

2022년 데이터 분석 청년인재 양성사업 일경험수련 결과보고서

식품안전정보원의 실무 개선을 위한 해외 제조업소 분류 알고리즘 구축

2022.01.31

Y220224	고수연
---------	-----

식품안전정보원

목 차

I . 일 경험 수련 개요	03
1. 참여기관 정보	03
2. 수행 기간	03
3. 일 경험 수련 주요 내용	03
4. 수행 체계	03
II . 업무수행 내용	04
1. 수행 개요	04
1.1 프로젝트명	04
2.2 수행 목표	04
3.3 문제	04
4.4 업무 활용방안	04
2. 수행 방법	04
2.1 데이터 탐색	04
2.2 전체 프로세스	06
2.3 프로세스 설계 세부 설명과 방법	06
3. 수행 결과	07
1. 자연어 처리	07
2. 지오큐딩	08
3. 수행 결과.	08
4. 기대효과	09
III . 기타 수행내용	09
1. 제조업소명 유사판단 시험과정	09
IV . 부록	11
1. 상세 코드	11

I. 일 경험 수련 개요

1. 참여기관 정보

- ☐ 기관 명 : 식품안전정보원
- ☐ 근무 조직: 식품안전정보원 정보화사업실 통합정부시스템부
- ☐ 기관 구분: 공공기관

2. 수행 기간

- ☐ 수련 기간: 2022.09.05. ~ 2023.01.31. (5개월)
- ☐ 세부 프로젝트 수행 기간
- 1) 해외제조업소 분류 알고리즘 구축: 2022.09.26. ~ 2022.10.11.
 - 2) 척척약사 앱 서비스 기획: 2022.10.12. ~ 2022.11.18.
 - 3) 감귤 착과량 예측 모델 구축: 2022.11.23. ~ 2022.12.14.
 - 4) 생산품종 분류 모델 구축: 2022.12.15. ~ 2023.01.18.

3. 일 경험 수련 주요 내용

- 1) 텍스트마이닝과 지오코딩을 활용하여 해외 제조업소를 분류하는 알고리즘 구축
- 2) 국민의 약제 복용 현황분석 및 고령자 대상의 척척약사 앱 서비스를 기획
- 3) 감귤 생산량 추정을 위한 나무 상태 기반, 감귤의 착과량 예측 모델 구축
- 4) 부정 유통을 차단하기 위한 개체정보를 기반, 생산품종을 구분 모델 구축

4. 수행 체계

- ☐ 수행원: 고수연, 이찬우
- ☐ 수행직무

고수연	이찬우
- 프로젝트 아이디어 제시 - 자료조사 - 분석 프로세스, 알고리즘 구성 - 모델 구축, 검증 - 보고서 정리	- 프로젝트 아이디어 제시 - 자료조사 - 분석 프로세스, 알고리즘 구성 - 모델 구축, 검증 - 보고서 정리

Ⅱ. 업무수행 내용

1. 수행 개요

1.1 프로젝트명

- ☐ 텍스트마이닝과 지오코딩을 활용한 해외 제조업소 분류 알고리즘 구축

1.2 수행 목표

- ☐ 해외 제조업소 관리 자동화
☐ 업소 정보를 기반으로 동일 업소 추출 및 분류

1.3 문제

- ☐ 제조업소 신청 작성 시 오타 등의 기입오류로 같은 업소가 여러 번 등록되는 문제
☐ 대표자 변경 및 사업장 이동 등의 재등록으로 같은 업소가 여러 번 등록되는 문제

1.4 업무 활용 방안

- ☐ 식품안전정보원에서는 수입식품 등을 국내로 수입하려는 자 또는 해외제조업소의 설치 운영자의 수입신고를 관리하고 있으며 2022년도 9월 기준, 95,045개의 정보가 등록되어있음
☐ 하지만 95,045개의 해외제조업소를 눈으로 비교하며 중복업소를 관리할 수 없음.
 알고리즘 구축을 통한 해외 제조업소 자동화로 업무 생산성을 현저히 향상시킬 수 있음

2. 수행 방법

2.1 데이터 탐색

가. 분석 데이터 목록

[표2-1] 분석 데이터 칼럼 리스트

파일명	칼럼명	형식	예시
20220926_ 해외제조업소 리스트 (등록, 수입중단).csv	제조업소코드	chr	AE000000114
	국가	chr	아랍에미리트
	지역	chr	두바이 토후국
	제조업소명	chr	ELLA ○○○ LIMITED
	제조업소 주소	chr	CENTRAL LAKE NO. ○○○ BY DUBAI, UAE.
	대표자명	chr	D○○○IZ
	전화번호	chr	9715689○○○
	이메일	chr	info@○○○.com
	등록일시	date time	2016/08/03 15:16:27

* 보안 상의 문제로 데이터 일부를 가림(이후 동일)

나. EDA 및 시각화

1) 중복 데이터

- ☐ 제조업소 코드를 제외한 나머지 항목들의 값이 완전히 동일한 데이터는 총 1개임

[표2-2] 중복 데이터 리스트

제조업소 코드	국가	지역	제조업소명	제조업소 주소	대표자명	전화번호	이메일	등록일시
VN000008504	베트남		○○○ Seafood Factor - ○○○ Corporation (SSC)	Area 13, ○○○, Binh An commune, Chau Thanh district, Kien Giang province	Nguyen ○○○ Hai	+849 191 ○○ 889	info@ ○○○. vn	2016/ 08/02 20:48: 53
VN000008520	베트남		○○○ Seafood Factory - ○○○ Corporation (SSC)	Area 13, ○○○, Binh An commune, Chau Thanh district, Kien Giang province	Nguyen ○○○ Hai	+849 191 ○○ 889	info@ ○○○. vn	2016/ 08/02 20:48: 53

2) 결측값 데이터

- ☐ 특정 칼럼에서 결측치가 발생하는 것을 확인함
- ☐ 지역 칼럼의 결측 데이터 개수는 4065개로 다른 항목에 비해 특히 높은 수치를 보임

[표2-3] 결측 데이터 개수

칼럼명	개수
제조업소코드	0
국가	0
지역	4065
제조업소명	0
제조업소 주소	1
대표자명	76
전화번호	301
이메일	234
등록일시	0

3) 그 외 세부적인 특징

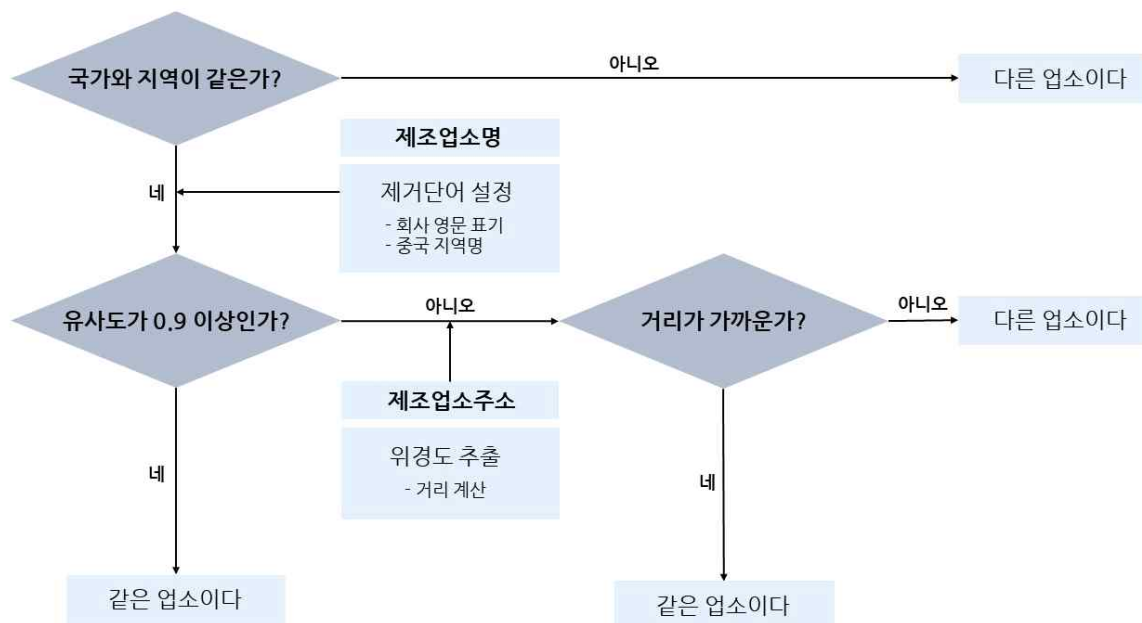
□ 제조업소명에 회사명 법인 영문 표기 약자가 들어가는 경우가 존재함
 'CO.LTD', 'PRODUCTS.CO', 'FZE', 'FZC', 'COOP', 'CORP', 'CO',
 'PTY LTD', 'LTDA', 'LTD', 'SAS', 'SAC', 'S.A', 'S.P.A', 'S.L',
 'B.V', 'SRL', 'LLC', 'L.L.C', 'INC', 'SP ZOO', 'SARL', 'SOOC',
 'GMBH'

□ 중국에 한하여 제조업소 명이 지역명으로 시작하는 경우가 다수 존재함

[표2-4] 중국 제조업소 명 예시

나라	지역	지역 영문	제조업소명
중국	광둥성	GUANGDONG	GUANGDONG ZHENGYI ○○○ STEEL INDUSTRY CO.,LTD
중국	푸젠성	FUJIAN	FUJIAN SHUANGYI ○○○ AND WOOD DEVELOPMENT CO., LTD
중국	지린성	JILIN	JILIN PROVINCE ○○○ WOOD CO., LTD
중국	톈진시	TIANJIN	TIANJIN TONGJIANG ○○○ & TRADE
중국	충칭시	CHONGQING	CHONGQING FENGDU ○○○ CO.,LTD.
중국	허베이성	HEBEI	HEBEI DELI ○○○ CO., LTD
중국	쓰촨성	SICHUAN	SICHUAN YUANKUN ○○○ FRUIT CO.,LTD
중국	하이난성	HAINAN	HAINAN ZHONGXIN ○○○ CHEMICAL CO.,LTD

2.2 전체 프로세스



[그림2-1] 프로세스

2.3 프로세스 설계 세부 설명과 방법

가. 데이터 정제방안

- 1) 중복데이터: 중복업소로 분리
- 2) 결측데이터: 제조업소 명과 제조업소 주소 중심으로 판단 불가로 분리
- 3) 유사도 계산을 방해하는 회사명 법인 영문 표기 약자와 지역명 제거

나. 텍스트마이닝을 이용한 업소명 유사도 계산

- 1) 제조업소명을 byte화하여 비교하였을 때 수치가 같으면 동일업소명을 가짐
- 2) 수치가 같지 않지만 유사할 경우 그 차이가 미세하다면 이 또한 동일업소로 판단함

□ SequenceMatcher

비교 문자	AHMAD TEA FZ LLS	AHMAD TEA FZ LLC
유사도 수치	0.9375	
비교 문자	AHMAD TEA FZ LL	AHMAD TEA FZ LLC
유사도 수치	0.967741935483871	
비교 문자	HYMOFRUIT LTD	HYMO FRUIT LTD
유사도 수치	0.9629629629629629	

다. 지오코딩을 이용한 주소지 거리계산

- 1) googlemaps 주소 기반으로 위경도 추출
- 2) 업소명이 높은 유사도를 보이는 두 제조업소의 주소지를 비교하여 거리 계산

□ Haversine

주소1	6A. AVENIDA 20-25 ZONA 10, EDIFICIO, PLAZA MARITIMA GUATEMALA
주소2	2DA. LOTE 69 COLONIA SANTA ISABEL II BENEFICIOSANTA ISABEL ZONA 6
거리 도출	8.44393461544122 (km)

3. 수행 결과

3.1 자연어 처리

가. 정규표현식

```
re.sub('[^\w\s]', '', inputString).strip()
```

- 문자열 집합에서 특수문자와 줄바꿈 등을 제거함

나. SequenceMatcher

```
import difflib
answer_bytes = bytes('AHMAD TEA FZ LL', 'utf-8')
answer_bytes_list = list(answer_bytes)

input_bytes = bytes('AHMAD TEA FZ LLC', 'utf-8')
input_bytes_list = list(input_bytes)

sm = difflib.SequenceMatcher(None, answer_bytes_list, input_bytes_list)
sm.ratio()
```

- 업소명을 byte화하여 비교함

```
difflib.SequenceMatcher(None, answer_bytes_list, input_bytes_list).ratio() > 0.90)
```

- 유사도가 0.9(최대:1) 이상인 제조업소를 분류함

3.2 지오코딩

가. 지오코딩

```
def getLoc(addr):
    try:
        geocode_result = gmaps.geocode(addr)
        loc = str(geocode_result[0]['geometry']['location'])
        loc_string = re.sub(r"[a-z: '{}]", "", loc).strip()
        lat, lng = list(map(float, loc_string.split(' ')))
        return (lat, lng)
    except:
        return None
```

- 데이터베이스에 올려져 있는 주소를 구글맵을 통해 검색하여 해당하는 건물의 위경도를 반환하고, 주소가 검색되지 않는 경우 None값 반환

나. 두 지점의 거리 계산 함수

```
# 위경도 거리 계산 함수
def distance(region1, region2):
    if (region1 != None)&(region2 != None):
        try:
            dist = haversine(getLoc(region1), getLoc(region2), unit = 'km')
            return dist
        except:
            return None
    else:
        return None
```

- 서로 다른 두 지점의 위경도를 입력하면 두 지점 사이의 거리를 계산하여 반환하는 함수

3.3 수행 결과

제조업소코드	제조업소명	유사코드1	유사수치1	거리(km)
GT000000152	EXPORTADOOO, S.A.	GT000000165	1	
GT000000141	AGRICOLA OOO,S.A.	GT000000266	1	
GT000000198	BENEFICIO OLAS TAPIAS	GT000000197	0.96969697	0
GT000000296	BENEFICIO OOOOL, S.A.	GT000000213	1	
GT000000213	BENEFICIO OOOOL	GT000000296	1	
GT000000197	BENEFICIO OOS TAPIAS	GT000000198	0.96969697	0
GT000000317	INMOBILIARIA OOO, S.A	GT000000310	1	
GT000000266	AGRICOLA OOO, S.A.	GT000000141	1	

4. 기대 효과

□ 데이터 오류율 감소

- 오타 등의 기입오류로 여러 번 등록된 업소 파악 가능
- 대표자 변경 및 사업장 이동 등의 재등록으로 여러 번 등록된 업소 파악 가능
- 주소 미기재 등의 등록형식에 맞지 않는 데이터 파악 가능
- 국가,지역과 주소가 일치하지 않는 업소 파악 가능

□ 주소지가 실존하는지의 여부 파악하여 파견 시 우선대상으로 선정하는데 도움을 줌

Ⅲ. 기타 수행내용

1. 제조업소명 유사판단 시험과정

1.1 비교 과정

가. n-gram

- n-gram이란 기존 문장이 있을 때 n개의 연속적인 단어 나열을 의미함
- 모든 단어의 빈도를 확인하는 것이 아니기 때문에 정확도가 떨어질 수 있음

```
def word_ngram(bow, num_gram):
    text = tuple(bow)
    ngrams = [text[x:x + num_gram] for x in range(0, len(text))]
    return tuple(ngrams)

def similarity(doc1, doc2):
    cnt = 0
    for token in doc1:
        if token in doc2:
            cnt = cnt + 1
```

```

return cnt/len(doc1)

#제거 리스트
remove_set = {'CO.LTD','PRODUCTS.CO','COOP','CORP','CO','PTY','LTDA','L
TD','SAS','SAC','S.A','S.P.A','S.L','B.V','SRL','LLC','INC','SP','ZOO','SARL','
SOOC'}

bow_std = 'AHMAD TEA FZ LLS'.split(' ')
bow_std = [i for i in bow_std if i not in remove_set]

doc_std = word_ngram(bow_std, 2)

i = 'AHMAD TEA FZ LLC'
bow_compare = i.split(' ')
bow_compare = [i for i in bow_compare if i not in remove_set]
doc_compare = word_ngram(bow_compare, 2)
similarity(doc_std, doc_compare) # 유사도 계산

```

나. 코사인 유사도

- ☐ 코사인 유사도는 문장 내의 각 단어들의 벡터를 구하고 벡터 간의 코사인 각도를 이용해 유사도를 판단함
- ☐ 두 벡터의 방향이 같을수록 유사하다고 판단하며 각도가 수직인 경우 0, 완전히 다를 경우 -1을 반환함

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf_vectorizer = TfidfVectorizer()

sentences = ('HYMOFRUIT LTD','HYMO FRUIT LTD')

tfidf_matrix = tfidf_vectorizer.fit_transform(sentences)
cos_similar = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])
cos_similar[0][0]

```

다. SequenceMatcher

```

import difflib
answer_bytes = bytes('AHMAD TEA FZ LL', 'utf-8')
answer_bytes_list = list(answer_bytes)
input_bytes = bytes('AHMAD TEA FZ LLC', 'utf-8')
input_bytes_list = list(input_bytes)
sm = difflib.SequenceMatcher(None, answer_bytes_list, input_bytes_list)
sm.ratio()

```

1.2 선정 결과

□ n-gram

비교 문자	AHMAD TEA FZ LLC	AHMAD TEA FZ LLC
유사도 수치	1.0	
비교 문자	AHMAD TEA FZ LLS	AHMAD TEA FZ LLC
유사도 수치	0.9375	
비교 문자	AHMAD TEA LL	AHMAD TEA FZ LLC
유사도 수치	1.0	

- 알파벳이 빠져도 단어 순서가 일정하면 동일하다 판단하는 문제가 발생하여 기각함

□ 코사인 유사도

비교 문자	HYMOFRUIT LTD	HYMO FRUIT LTD
유사도 수치	0.26055567105626243	

- 띄어쓰기를 오타로 인지하지 못하는 문제가 발생하여 기각함

□ SequenceMatcher

비교 문자	AHMAD TEA FZ LLS	AHMAD TEA FZ LLC
유사도 수치	0.9375	
비교 문자	AHMAD TEA FZ LL	AHMAD TEA FZ LLC
유사도 수치	0.967741935483871	
비교 문자	HYMOFRUIT LTD	HYMO FRUIT LTD
유사도 수치	0.9629629629629629	

- 띄어쓰기, 알파벳 누락, 오타 모두 잘 고려하여 해당 방법을 채택함

IV. 부록

1. 상세 코드

이 코드를 처음 사용하는 PC라면 시작하기에 앞서 설치해야할 라이브러리 !pip install difflib #유사업소명 분류 시 사용 !pip install -U googlemaps #위경도 추출 시 사용 !pip install haversine #주소지 거리계산 시 사용
메모리 정리 import torch, gc gc.collect() torch.cuda.empty_cache()
파일 불러오기 import pandas as pd df= pd.read_excel('20220926_해외제조업소 리스트(등록, 수입중단).xlsx',header=0) # 해당 파일은 소스코드 제작시 사용한 파일명이므로 실행 시 수정 요망
import re # 제거단어 설정 remove_set = {'CO.LTD','PRODUCTS.CO','FZE','FZC','COOP','CORP','CO','PTY LTD','LTD A','LTD','SAS','SAC','S.A','S.P.A','S.L','B.V','SRL','LLC','L.L.C','INC','SP ZOO','SARL','S

OOO', 'GMBH', 'COMPANY LIMITED', 'FACTORY'}

중국 도시명

china_city = ['HONGKONG', 'MACAU', 'BEIJING', 'CHONGQING', 'SHANGHAI', 'TIANJIN', 'A
N QING', 'BENGBU', 'BOZHOU', 'CHAOHU', 'CHIZHOU', 'CHUZHOU', 'FUYANG', 'GUANGDE
, 'HEFEI', 'HUAIBEI', 'HUAINAN', 'HUANGSHAN', 'JIESHOU', 'LUAN', 'MAANSHAN', 'MING
GUANG', 'NINGGUO', 'QIANSHAN', 'SUZHOU', 'TIANCHANG', 'TONGCHENG', 'TONGLING'
, 'WUHU', 'WUWEI', 'XUANCHENG', 'FUAN', 'FUDING', 'FUQING', 'FUZHOU', 'JIANOU', 'JIN
JIANG', 'LONGHAI', 'LONGYAN', 'NANAN', 'NANPING', 'NINGDE', 'PUTIAN', 'QUANZHOU',
'SANMING', 'SHAOWU', 'SHISHI', 'WUYISHAN', 'XIAMEN', 'YONGAN', 'ZHANGPING', 'ZHA
NGZHOU', 'BAIYIN', 'DINGXI', 'DUNHUANG', 'HEZUO', 'HUATING', 'JIAYUGUAN', 'JINCHA
NG', 'JIUQUAN', 'LANZHOU', 'LINXIA', 'LONGNAN', 'PINGLIANG', 'QINGYANG', 'TIANSHUI
, 'WUWEI', 'YUMEN', 'ZHANGYE', 'CHAOZHOU', 'DONGGUAN', 'ENPING', 'FOSHAN', 'GAO
ZHOU', 'GUANGZHOU', 'HESHAN', 'HEYUAN', 'HUAZHOU', 'HUIZHOU', 'JIANGMEN', 'JIEY
ANG', 'KAIPING', 'LECHANG', 'LEIZHOU', 'LIANJIANG', 'LIANZHOU', 'LUFENG', 'LUODING'
, 'MAOMING', 'MEIZHOU', 'NANXIONG', 'PUNING', 'QINGYUAN', 'SHANTOU', 'SHANWEI',
'SHAOGUAN', 'SHENZHEN', 'SIHUI', 'TAISHAN', 'WUCHUAN', 'XINGNING', 'XINYI', 'YANG
CHUN', 'YANGJIANG', 'YINGDE', 'YUNFU', 'ZHANJIANG', 'ZHAOQING', 'ZHONGSHAN', 'ZH
UHAI', 'BAISE', 'BEIHAI', 'BEILIU', 'CENXI', 'CHONGZUO', 'DONGXING', 'FANGCHENGGA
G', 'GUIGANG', 'GUILIN', 'GUIPING', 'HECHI', 'HENGZHOU', 'HESHAN', 'HEZHOU', 'JINGXI'
, 'LAIBIN', 'LIPU', 'LIUZHOU', 'NANNING', 'PINGGUO', 'PINGXIANG', 'QINZHOU', 'WUZHOU
, 'YULIN', 'ANSHUN', 'BIJIE', 'CHISHUI', 'DUYUN', 'FUQUAN', 'GUIYANG', 'KAILI', 'LIUPA
NSHUI', 'PANZHOU', 'QIANXI', 'QINGZHEN', 'RENHUI', 'TONGREN', 'XINGREN', 'XINGYI',
'ZUNYI', 'DANZHOU', 'DONGFANG', 'HAIKOU', 'QIONGHAI', 'SANSHE', 'SANYA', 'WANNI
NG', 'WENCHANG', 'WUZHISHAN', 'ANGUO', 'BAODING', 'BAZHOU', 'BOTOU', 'CANGZHO
U', 'CHENGDE', 'DINGZHOU', 'GAOBEIDIAN', 'HANDAN', 'HENGSHUI', 'HEJIAN', 'HUANGH
UA', 'JINZHOU', 'LANGFANG', 'LUANZHOU', 'NANGONG', 'PINGQUAN', 'QIANAN', 'QINHU
ANGDAO', 'RENQIU', 'SANHE', 'SHAHE', 'SHENZHOU', 'SHIJIAZHOU', 'TANGSHAN', 'XI
NJI', 'WUAN', 'XINGTAI', 'XINLE', 'ZHANGJIAKOU', 'ZHUOZHOU', 'ZUNHUA', 'ANDA', 'BEI
AN', 'DAQING', 'DONGNING', 'FUJIN', 'FUYUAN', 'HAILIN', 'HAILUN', 'HARBIN', 'HEGANG'
, 'HEIHE', 'HULIN', 'JIAMUSI', 'JIXI', 'MISHAN', 'MOHE', 'MUDANJIANG', 'MULING', 'NEHE',
'NENJIANG', 'NINGAN', 'QIQIHAR', 'QITAIHE', 'SHANGZHI', 'SHUANGYASHAN', 'SUIFENH
E', 'SUIHUA', 'TIELI', 'TONGJIANG', 'WUCHANG', 'WUDALIANCHI', 'YICHUN', 'ZHAODONG
, 'ANYANG', 'CHANGGE', 'CHANGYUAN', 'DENGFEI', 'DENGZHOU', 'GONGYI', 'HEBI', 'H
UIXIAN', 'JIAOZUO', 'JIYUAN', 'KAIFENG', 'LINGBAO', 'LINZHOU', 'LUOHE', 'LUOYANG',
'MENGZHOU', 'NANYANG', 'PINGDINGSHAN', 'PUYANG', 'QINYANG', 'RUZHOU', 'SANME
NXIA', 'SHANGQIU', 'WEIHUI', 'WUGANG', 'XIANGCHENG', 'XINGYANG', 'XINMI', 'XINXIA
NG', 'XINYANG', 'XINZHENG', 'XUCHANG', 'YANSHI', 'YIMA', 'YONGCHENG', 'YUZHOU',
'ZHENGZHOU', 'ZHOUKOU', 'ZHUMADIAN', 'ANLU', 'CHIBI', 'DANGYANG', 'DANJIANGKOU
, 'DAYE', 'ENSHI', 'EZHO', 'GUANGSHUI', 'HANCHUAN', 'HONGHU', 'HUANGGANG', 'H
UANGSHI', 'JIANLI', 'JINGMEN', 'JINGSHAN', 'JINGZHOU', 'LAOHEKOU', 'LICHUAN', 'MAC
HENG', 'QIANJIANG', 'SHISHOU', 'SHIYAN', 'SUIZHOU', 'SONGZI', 'TIANMEN', 'WUHAN',
WUXUE', 'XIANGYANG', 'XIANNING', 'XIAOTAO', 'XIAOGAN', 'YICHANG', 'YICHENG', 'YI
DU', 'YINGCHENG', 'ZAOYANG', 'ZHIJIANG', 'ZHONGXIANG', 'CHANGDE', 'CHANGNING',
CHANGSHA', 'CHENZHOU', 'HENGYANG', 'HONGJIANG', 'HUAHUA', 'JINSHI', 'JISHOU', 'L
EIJANG', 'LENGSHUIJIANG', 'LIANYUAN', 'LILING', 'LINXIANG', 'LIUYANG', 'LOUDI', 'MIL
UO', 'NINGXIANG', 'SHAOSHAN', 'SHAODONG', 'SHAOYANG', 'WUGANG', 'XIANGTAN', 'X
IANGXIANG', 'YIYANG', 'YONGZHOU', 'YUANJIANG', 'YUEYANG', 'ZHANGJIAJIE', 'ZHUZH
OU', 'ZIXING', 'ARXAN', 'BAOTOU', 'BAYANNUR', 'CHIFENG', 'ERENHOT', 'ERGUN', 'FEN
GZHEN', 'GENHE', 'HOHHOT', 'HOLINGOL', 'HULUNBUIR', 'MANZHOU', 'ORDOS', 'TONG

LIAO', 'ULANHOT', 'ULANQAB', 'WUHAI', 'XILINHOT', 'YAKESHI', 'ZHALANTUN', 'CHANGSHU', 'CHANGZHOU', 'DANYANG', 'DONGTAI', 'GAOYOU', 'HAIAN', 'HUAIAN', 'JIANGYIN', 'JINGJIANG', 'JURONG', 'LIYANG', 'LIANYUNGANG', 'KUNSHAN', 'NANJING', 'NANTONG', 'PIZHOU', 'QIDONG', 'RUGAO', 'SUQIAN', 'SUZHOU', 'TAICANG', 'TAIXING', 'TAIZHOU', 'WUXI', 'XINGHUA', 'XINYI', 'XUZHOU', 'YANCHENG', 'YANGZHONG', 'YANGZHOU', 'YIXING', 'YIZHENG', 'ZHANGJIAGANG', 'ZHENJIANG', 'DEXING', 'FENGCHENG', 'FUZHOU', 'GANZHOU', 'GAOAN', 'GONGQINGCHENG', 'GUIXI', 'JIAN', 'JINGDEZHEN', 'JINGGANGSHAN', 'JIUJIANG', 'LEPING', 'LONGNAN', 'LUSHAN', 'NANCHANG', 'PINGXIANG', 'RUICHANG', 'RUJIN', 'SHANGRAO', 'XINYU', 'YICHUN', 'YINGTAN', 'ZHANGSHU', 'BAICHENG', 'BAISHAN', 'CHANGCHUN', 'DAAN', 'DEHUI', 'DUNHUA', 'FUYU', 'GONGZHULING', 'HELONG', 'HUADIAN', 'HUNCHUN', 'JIAN', 'JIAOHE', 'JILIN', 'LIAOYUAN', 'LINJIANG', 'LONGJING', 'MEIHEKOU', 'PANSI', 'SHUANGLIAO', 'SHULAN', 'SIPING', 'SONGYUAN', 'TAONAN', 'TONGHUA', 'TUMEN', 'YANJI', 'YUSHU', 'ANSHAN', 'BENXI', 'BEIPIAO', 'BEIZHEN', 'CHAOYANG', 'DALIAN', 'DANDONG', 'DASHIQIAO', 'DENGTA', 'DIAOBINGSHAN', 'DONGGANG', 'FENGCHENG', 'FUSHUN', 'FUXIN', 'GAIZHOU', 'HAICHENG', 'HULUDAO', 'JINZHOU', 'KAIYUAN', 'LIAOYANG', 'LINGHAI', 'LINGYUAN', 'PANJIN', 'SHENYANG', 'TIELING', 'WAFANGDIAN', 'XINGCHENG', 'XINMIN', 'YINGKOU', 'ZHUANGHE', 'GUYUAN', 'LINGWU', 'QINGTONGXIA', 'SHIZUISHAN', 'WUZHONG', 'YINCHUAN', 'ZHONGWEI', 'DELINGHA', 'GOLMUD', 'HAIDONG', 'MANGNAI', 'TONGREN', 'XINING', 'YUSHU', 'ANKANG', 'BAOJI', 'BINZHOU', 'HANCHENG', 'HANZHONG', 'HUAYIN', 'SHANGLUO', 'SHENMU', 'TONGCHUAN', 'WEINAN', 'XIAN', 'XIANYANG', 'XINGPING', 'XUNYANG', 'YANAN', 'YULIN', 'ZICHANG', 'ANQIU', 'BINZHOU', 'CHANGYI', 'DEZHOU', 'DONGYING', 'FEICHENG', 'GAOMI', 'HAIYANG', 'HEZE', 'JIAOZHOU', 'JINAN', 'JINING', 'LAIXI', 'LAIYANG', 'LAIZHOU', 'LELING', 'LIAOCHENG', 'LINQING', 'LINYI', 'LONGKOU', 'PINGDU', 'QINGDAO', 'QINGZHOU', 'QIXIA', 'QUFU', 'RIZHAO', 'RONGCHENG', 'RUSHAN', 'SHOUGUANG', 'TAIAN', 'TENGZHOU', 'WEIFANG', 'WEIHAI', 'XINTAI', 'YANTAI', 'YUCHENG', 'ZAOZHUANG', 'ZHAOYUAN', 'ZHUCHENG', 'ZIBO', 'ZOUCHENG', 'ZOUPIPING', 'CHANGZHI', 'DATONG', 'FENYANG', 'GAOPING', 'GUJIAO', 'HEJIN', 'HOUMA', 'HUAIREN', 'HUOZHOU', 'JIEXIU', 'JINCHENG', 'JINZHONG', 'LINFEN', 'LÜLIANG', 'SHUOZHOU', 'TAIYUAN', 'XIAOYI', 'XINZHOU', 'YANGQUAN', 'YONGJI', 'YUNCHENG', 'YUANPING', 'BARKAM', 'BAZHONG', 'CHENGDU', 'CHONGZHOU', 'DAZHOU', 'DEYANG', 'DUJIANGYAN', 'EMEISHAN', 'GUANGAN', 'GUANGHAN', 'GUANGYUAN', 'HUAYING', 'HUILI', 'JIANGYOU', 'JIANYANG', 'KANGDING', 'LANGZHONG', 'LESHAN', 'LONGCHANG', 'LUZHOU', 'MIANZHU', 'MEISHAN', 'MIANYANG', 'NANCHONG', 'NEIJIAN', 'PANZHIHUA', 'PENGGZHOU', 'QIONGLAI', 'SHIFANG', 'SHEHONG', 'SUINING', 'WANYUAN', 'XICHANG', 'YANAN', 'YIBIN', 'ZIGONG', 'ZIYANG', 'LHASA', 'NAGQU', 'NYINGCHI', 'QAMDO', 'SHANNAN', 'XIGAZÊ', 'AKSU', 'ALASHANKOU', 'ALTAY', 'ARAL', 'ARTUX', 'BEITUN', 'BOLE', 'CHANGJI', 'FUKANG', 'HAMI', 'HOTAN', 'HUYANGHE', 'KARAMAY', 'KASHGAR', 'KHORGAS', 'KOKDALA', 'KORLA', 'KUNYU', 'KUQA', 'KUYTUN', 'SHAWAN', 'SHIHEZI', 'SHUANGHE', 'TACHENG', 'TIEMENGUAN', 'TUMXUK', 'TURPAN', 'ÜRÜMQI', 'WUJIAQU', 'WUSU', 'XINXING', 'YINING', 'ANNING', 'BAOSHAN', 'CHENGJIANG', 'CHUXIONG', 'DALI', 'GEJIU', 'JINGHONG', 'KAIYUAN', 'KUNMING', 'LINCANG', 'LIJIANG', 'LUFENG', 'LUSHUI', 'MANGSHI', 'MENGZI', 'MILE', 'PUER', 'QUJING', 'RUILI', 'SHANGRI-LA', 'SHUIFU', 'TENGCHONG', 'WENSHAN', 'XUANWEI', 'YUXI', 'ZHAOTONG', 'CIXI', 'DONGYANG', 'HAINING', 'HANGZHOU', 'HUZHOU', 'JIANDE', 'JIANGSHAN', 'JIAXING', 'JINHUA', 'LANXI', 'LINHAI', 'LISHUI', 'LONGGANG', 'LONGQUAN', 'NINGBO', 'PINGHU', 'QUZHOU', 'RUIAN', 'SHAOXING', 'SHENGZHOU', 'TAIZHOU', 'TONGXIANG', 'WENLING', 'WENZHOU', 'YIWU', 'YONGKANG', 'YUEQING', 'YUHUAN', 'YUYAO', 'ZHOUZHAN', 'ZHUJI', 'CHAOAN']

업소명 정제 함수 정의

```
def clean_text(inputString):
```

```

for i in remove_set:
    inputString = inputString.replace(i, '').strip()

text_rmv = re.sub('[^\w\s]', '', inputString).strip()
return text_rmv

# 중국 도시명 정제 함수
def clean_text_china(inputString):
    for i in china_city:
        inputString = inputString.replace(i+ ' ', '').rstrip()

    text_rmv = re.sub('[^\w\s]', '', inputString).strip()
    return text_rmv

# 제조업소명 정제 칼럼 추가
df['제조업소명_정제'] = [clean_text(i) for i in df['제조업소명']]

# 중국 제조업소명 추가 정제
for i in df.index:
    if df['국가'][i] == '중국':
        df['제조업소명_정제'][i] = clean_text_china(df['제조업소명_정제'][i])

import googlemaps

# 구글 key 설정
# 해당 키는 소스코드 제작시 임시로 발급한 key로 추후 삽입해야한다
gmaps = googlemaps.Client(key='AIzaSyB6lI1FiNW4eyt2dn4jZdETYESDmvHjjl0')

# 위경도 변환 함수
def getLoc(addr):
    try:
        geocode_result = gmaps.geocode(addr)
        loc = str(geocode_result[0]['geometry']['location'])
        loc_string = re.sub(r'[a-z:']', '', loc).strip()
        lat, lng = list(map(float, loc_string.split(' ')))
        return (lat, lng)
    except:
        return None
from haversine import haversine

# 위경도 거리 계산 함수
def distance(region1, region2):

```

```

if (region1 != None)&(region2 != None):
    try:
        dist = haversine(getLoc(region1),getLoc(region2), unit = 'km')
        return dist
    except:
        return None
    else:
        return None

# 유사업소 데이터 추출
import difflib
from tqdm import notebook

li_similar_total = [] # 유사업소코드
li_dist_total = [] # 유사업소코드 간 실제 거리
li_sim_total = [] # 유사정도 수치값

for i in notebook.tqdm(df.index):
    li_similar_Seq = []
    li_dist_Seq = []
    li_sim_Seq = []
    answer_bytes_list = list(bytes(df['제조업소명_정제'][i], 'utf-8'))

    df_sample=df[(df['국가']==df['국가'][i])&(df['지역']==df['지역'][i])]

    for j in df_sample.index:
        input_bytes_list = list(bytes(df_sample['제조업소명_정제'][j], 'utf-8'))

        if (df['제조업소코드'][i] != df_sample['제조업소코드'][j])&(difflib.SequenceMatcher(None, answer_bytes_list, input_bytes_list).ratio() > 0.90):
            li_similar_Seq.append(df_sample['제조업소코드'][j])
            li_sim_Seq.append(difflib.SequenceMatcher(None, answer_bytes_list, input_bytes_list).ratio())
            li_dist_Seq.append(distance(df['제조업소 주소'][i],df_sample['제조업소 주소'][j]))

    li_similar_total.append(li_similar_Seq)
    li_sim_total.append(li_sim_Seq)
    li_dist_total.append(li_dist_Seq)

# 1시간 당 1만건 처리
# 데이터프레임 생성
max_cnt = [len(i) for i in li_similar_total]
columns_name = ['유사코드'+str(i+1) for i in range(max(max_cnt))]

```

```

columns_sim = ['유사수치'+str(i+1) for i in range(max(max_cnt))]
columns_dist = ['실제거리'+str(i+1) for i in range(max(max_cnt))]
smilar_name=pd.DataFrame(li_similar_total, columns=columns_name)
similarity=pd.DataFrame(li_sim_total, columns=columns_sim)
dist_cal=pd.DataFrame(li_dist_total, columns=columns_dist)

# 반복해서 칼럼 추가
df_final = df.copy()
for i in range(smilar_name.shape[1]):
    df_final=df_final.join(smilar_name[smilar_name.columns[i]], how='left')
    df_final=df_final.join(similarity[similarity.columns[i]], how='left')
    df_final=df_final.join(dist_cal[dist_cal.columns[i]], how='left')

# 유사한 업체만 따로 추출
df_similar = df_final.dropna(subset = ['유사코드1'])
df_similar.reset_index(drop=True,inplace=True)

# 정렬
df_similar = df_similar.sort_values('유사수치1',ascending=False)
df_similar.sort_values(by=['국가', '지역', '제조업소명'], inplace=True)

df_similar.drop(['제조업소명_정제'],axis=1,inplace=True)
df_similar.reset_index(drop=True,inplace=True)
df_final.drop(['제조업소명_정제'],axis=1,inplace=True)

# csv로 저장
df_final.to_csv('total_check.csv',encoding='cp949',index=0) #전체 데이터
df_similar.to_csv('similarity_check.csv',encoding='cp949',index=0) #유사코드 데이터

```